

OOP

目录

一、考试范围	2
二、课程知识框架	2
三、各章详解	2
第 1 章面向对象程序设计基础	2
本章总览	2
一、纯虚函数与抽象类	3
1.1 纯虚函数 必背	3
1.2 纯虚析构函数 重点	4
二、向下类型转换	5
2.1 向下类型转换的必要性与安全性 重点	5
2.2 dynamic_cast 与 static_cast 对比 高频	6
三、多重继承中的虚函数	8
多重继承与抽象接口 重点	8
四、多态	9
4.1 多态的概念与条件 必背	9
4.2 动态多态与静态多态对比 高频	10
五、函数模板与类模板	11
5.1 函数模板与类型推导 必背	11
5.2 类模板与非类型参数 重点	13
本章小结	15

> 本笔记由 [复习资料整理 Agent] 自动生成：入口无损读取全部原始资料，出口按考试优先原则整理。

一、考试范围

（资料中未显式标注考试范围，以下为自动识别的章节范围，请以教师最终通知为准）

- 面向对象程序设计基础（OOP）

二、课程知识框架

OOP

- 面向对象程序设计基础

（OOP）

- 上期要点回顾
- 本讲内容提要
- 纯虚函数
- 抽象类
- 纯虚函数与抽象类示例
- 纯虚析构函数
- 回顾：向上类型转换
- 向下类型转换
- 示例
- 类型转换其他用法
- 向上向下类型转换与虚函数表
- 回忆：多重继承

三、各章详解

第 1 章面向对象程序设计基础

（OOP）

本章总览

本章核心围绕 C++ 面向对象的两大支柱：**多态与模板**。首先，我们通过纯虚函数与抽象类来定义清晰的“接口”，这是实现多态的基础。其次，利用 `dynamic_cast` 进行安全的向下类型转换，访问派生类特有接口。接着，我们探讨多态的核心：通过基类指针或引用来调用虚函数，实现程序行为的动态变化，并引出了静态多态（模板）与动态多态的对比。最后，我们学习函数模板和类模板，它们实现了与类型无关的代码复用，是编译期多态的关键。考试的重点在于区分多态的条件、`dynamic_cast` 与 `static_cast` 的差异，以及模板的使用与原理。

高频考点

1. 纯虚函数与抽象类的定义、特点及使用场景。

2. `dynamic_cast` 和 `static_cast` 的区别与用法，特别是向下类型转换的安全性问题。
3. 产生多态效果的必要条件（继承、虚函数、指针/引用）。
4. 函数模板与类模板的定义、实例化及非类型参数。
5. 动态多态（运行时绑定）与静态多态（编译时绑定）的概念对比。

方法清单

- 定义接口：声明纯虚函数将类变为抽象类。
- 安全向下转换：使用 `dynamic_cast` 并检查指针是否为 `nullptr`（或捕获 `bad_cast` 异常）。
- 实现多态：基类指针/引用调用 `virtual` 函数。
- 实现代码复用：定义函数模板或类模板，将类型参数化。

一、纯虚函数与抽象类

1.1 纯虚函数 必背

1. 定义

- 纯虚函数：在虚函数声明的末尾加上 `= 0`，即构成纯虚函数。

```
virtual ReturnType functionName(ParameterList) = 0;
```

- 抽象类：含有至少一个纯虚函数的类。它不能实例化对象，主要用途是为派生类规定必须实现的“接口”。

1. 原理与推导

- 设计意图：有些基类在设计时，其行为本身无法具体实现（如“动物”的“叫”），此时应将其行为函数声明为纯虚函数，强制每个派生类根据自己的特性去实现它。
- 接口规范：抽象类为所有派生类创建了一个公共的“契约”，任何继承抽象类的派生类（除纯虚析构函数外）都必须实现这些纯虚函数，否则派生类自身也将成为抽象类。
- 对象切片规避：因抽象类无法创建对象，有效避免了向上类型转换时的对象切片问题，确保只有指针和引用能用于多态。

2. 易错点

- 错误观念：纯虚函数不能有实现。
 - 纠正：纯虚函数可以有函数体，可以在类外定义，为所有派生类提供默认行为，但派生类仍需显式覆盖。派生类可以通过 `BaseClassName::FuncName()` 调用该默认实现。
- 继承与抽象：

- 如果派生类没有覆盖基类中**所有**纯虚函数，那么该派生类依然是一个抽象类，不能创建对象。
- **唯一例外**：纯虚析构函数，即使派生类未显式实现，编译器也会自动合成，所以不影响派生类实例化。

1.2 纯虚析构函数 **重点**

1. 定义

- 析构函数被声明为纯虚函数。

```
class Base {  
public:  
    virtual ~Base() = 0; // 纯虚析构函数  
};
```

- **特殊规则**：纯虚析构函数**必须**提供函数体定义（因为派生类析构时会调用它），这与普通纯虚函数不同。

```
Base::~~Base() { /* 必须有函数体 */ }
```

1. 原理与推导

- **目的**：当某个类只想作为抽象接口，但所有成员函数都有合适的非纯虚实现时，可将析构函数设为纯虚函数，使类成为抽象类，防止用户直接创建该类的对象。
- **派生类的行为**：即使派生类没有显式定义析构函数，编译器合成的默认析构函数也是非纯虚的，满足条件，因此派生类可以被实例化。这与普通纯虚函数的规则完全不同。

2. **易错点**

特性	普通纯虚函数	纯虚析构函数
语法	<code>virtual void f() = 0;</code>	<code>virtual ~Base() = 0;</code>
函数体	可选，可以提供默认实现	必须在类外提供定义
派生类未实现	派生类仍为抽象类	编译器会合成默认析构函数，派生类不是抽象类，可以实例化

1. 例题

例题

题目

指出下面代码中的错误并给出理由。（课件 P12 改编）

```
#include <iostream>
class Base {
public:
    virtual ~Base() = 0; // 声明纯虚析构函数
};
// 缺少 Base::~~Base() {} 的定义

class Derive : public Base {}; // 派生类未显式实现析构函数

int main() {
    Derive d1; // 这行能编译通过吗?
    return 0;
}
```

例题

思路

1. 确认 Base 是抽象类。2. 判断纯虚析构函数是否提供了定义。3. 判断派生类 Derive 是否可以实例化。

解答

1. 代码会导致链接错误，因为纯虚析构函数 Base::~~Base() 只有声明，没有定义。派生类 Derive 的对象 d1 在析构时会调用基类的析构函数，而该函数体未定义。2. 即使添加了 Base::~~Base() {} 的定义，Derive d1; 这行也能成功编译。虽然基类是抽象类，但因为纯虚析构函数的特殊规则，编译器为 Derive 合成的析构函数是非纯虚的，使得 Derive 不再是无法实例化的抽象类。

技巧

纯虚析构函数必须提供定义是它和普通纯虚函数最核心的区别。它的特殊性在于，它能把一个类变成抽象类，同时又不要求派生类必须自定义析构函数。

二、向下类型转换

2.1 向下类型转换的必要性与安全性 **重点**

1. 定义

- **向下类型转换**：将基类指针或引用，转换为派生类指针或引用的操作。这是在类继承层次中向下移动。
- **目的**：当我们使用基类指针（向上类型转换后）统一管理派生类对象时，会丢失派生类的特有行为（特性）。如果需要调用这些特性，就必须先进行安全的类型识别和转换。

2. 原理与推导

- **安全性问题：**一个基类指针可能指向多种不同的派生类对象。如果我们将一个指向 A 类对象的基类指针，错误地转换为 B 类指针，就会导致非法内存访问。
- **RTTI 机制：**C++ 的运行时类型识别（Run-Time Type Information, RTTI）允许程序在运行时获取对象的实际类型。`dynamic_cast` 正是利用 RTTI 来确保转换的安全性。

3. 解题步骤

- 使用 `dynamic_cast` 进行安全转换：
 - (a) **前提条件：**被转换的源类型（T1）必须是**多态类型**，即它声明或继承了至少一个虚函数。目标类型（T2）没有此要求。
 - (b) **指针转换：**`T2* ptr = dynamic_cast<T2*>(base_ptr);`
 - **结果检查：**转换成功后，`ptr` 指向目标类型对象；若转换失败（类型不匹配），`ptr` 为 `nullptr`。务必检查返回值。

```
if (ptr != nullptr) {  
    // 安全使用 ptr  
}
```

1. 引用转换：`T2& ref = dynamic_cast<T2&>(base_ref);`

- **异常处理：**转换失败时，会抛出 `std::bad_cast` 异常。

```
try {  
    T2& ref = dynamic_cast<T2&>(base_ref);  
    // 安全使用 ref  
} catch (const std::bad_cast& e) { /* 处理转换失败 */ }
```

2.2 `dynamic_cast` 与 `static_cast` 对比 高频

1. 定义与对比

特性	<code>dynamic_cast</code>	<code>static_cast</code>
工作时期	运行时 (利用 RTTI)	编译时
安全检查	安全，运行时检查对象真实类型。	不安全，只检查继承关系，不保证指向目标的正确性。
性能	较慢，有运行时开销。	较快，无运行时开销。
使用前提	源类型必须是多态类型。	源和目标间必须有明确的继承关系。
失败表现	指针转换返回 <code>nullptr</code> ；引用转换抛出 <code>bad_cast</code> 异常。	无任何提示，可能导致非法内存访问或未定义行为。
典型用途	向下类型转换 (Downcasting)。	向上类型转换 (Upcasting, 尽管不必要), 其他有定义的转换。

1. 易错点

- 错误使用 `static_cast` 向下转换：这是最常见的错误。即使指向的是错误的派生类对象，`static_cast` 也会成功返回一个非空指针，但访问其成员会导致未定义行为。
- 对象转换 vs 指针/引用转换：`dynamic_cast` 和 `static_cast` 都不能对非多态类型的对象进行向下转换。`dynamic_cast` 只接受指针或引用。
- 使用 `dynamic_cast` 检查失败：转换失败返回 `nullptr`，不检查直接解引用是常见错误。

2. 例题

例题

题目

分析以下代码，说明 `static_cast` 和 `dynamic_cast` 向下转型时的区别。（课件 P17-18 综合）

```
#include <iostream>

class B { public: virtual void f() {} };
class D : public B { public: int i{2018}; };

int main() {
    B b;
    D d;
    B* pb = &d; // 指向D对象的基类指针

    // Case 1: static_cast, 基类指针实际指向派生类对象
    D* pd1 = static_cast<D*>(pb);
    std::cout << "pd1->i = " << pd1->i << std::endl; // 期望输出2018

    // Case 2: static_cast, 基类指针指向基类对象
    D* pd2 = static_cast<D*>(&b);
    std::cout << "pd2->i = " << pd2->i << std::endl; // 未定义行为

    // Case 3: dynamic_cast, 基类指针实际指向派生类对象
    D* pd3 = dynamic_cast<D*>(pb);
    std::cout << "pd3->i = " << pd3->i << std::endl; // 期望输出2018

    // Case 4: dynamic_cast, 基类指针指向基类对象
    D* pd4 = dynamic_cast<D*>(&b);
    if (pd4 == nullptr) {
        std::cout << "pd4 is nullptr" << std::endl; // 安全地检测到失败
    }
    return 0;
}
```

例题

思路

1. 明确 `pb` 指向 `d`，类型为 `D`；`&b` 指向 `b`，类型为 `B`。2. 对于 `static_cast`，编译器只在编译期检查 `B` 和 `D` 是否有继承关系。有则通过，不检查运行时类型。3. 对于 `dynamic_cast`，它会在运行时检查指针（或引用）指向对象的真实类型是否与目标类型兼容。

解答

1. `pd1` 的转换：`static_cast` 发现 `B` 和 `D` 是继承关系，编译通过。运行时 `pb` 实际指向 `D` 对象，转换成功，`pd1->i` 输出 2018。2. `pd2` 的转换：`static_cast` 同样编译通过。但 `pd2` 实际指向一个 `B` 对象，访问其不存在的成员 `i` 将导致未定义行为，可能输出乱码。3. `pd3` 的转换：`dynamic_cast` 在运行时检查 `pb` 实际指向的是 `D` 对象，检查通过，转换成功，`pd3->i` 输出 2018。4. `pd4` 的转换：`dynamic_cast` 检查 `&b` 实际指向的是 `B` 对象，不能转换为 `D*`，转换失败，返回 `nullptr`。程序通过检查 `if (pd4 == nullptr)` 安全地处理了此情况。

技巧

当需要进行向下类型转换时，始终优先使用 `dynamic_cast`，并养成检查返回值是否有效（指针）或捕获异常（引用）的习惯。只有在能够 100% 确定运行时类型正确无误时，才可使用性能更高的 `static_cast`。

三、多重继承中的虚函数

多重继承与抽象接口 重点

1. 定义

- 多重继承：一个类可以从多个基类继承。这带来了二义性、钻石型继承等问题。
- 接口继承：在 C++ 中，类型“接口”的概念通常通过只包含纯虚函数的抽象类来实现。

2. 原理与推导

- 最佳实践：为了解决多继承的复杂性，一个经典的原则是“最多继承一个非抽象类（is-a），可以继承多个抽象类（接口）”。这样既避免了数据冗余和二义性，又使一个类能同时扮演多种角色（实现多个接口）。
- 示例推理：如课件示例，`Human` 类同时继承 `WhatCanSpeak` 和 `WhatCanMotion` 这两个纯接口（抽象类）。这使得 `Human` 对象可以被 `doSpeak` 和 `doMotion` 接受，满足了“一个对象对应多个接口”的设计需求，且没有引入数据成员上的二义性。

3. 例题

例题

题目

说明多重继承中 Human 类对象可以被 doSpeak 和 doMotion 函数接受的原因。(课件 P26 分析)

思路

1. 检查两个函数的参数类型。2. 检查 Human 类与这些参数类型的关系。3. 运用向上类型转换和多态的知识。

解答

1. 函数 doSpeak 的形参类型是 WhatCanSpeak*, 函数 doMotion 的形参类型是 WhatCanMotion*。2. Human 类通过多重继承, 公有派生自 WhatCanSpeak 和 WhatCanMotion, 形成了 is-a 关系: 一个 Human 对象也是一个 WhatCanSpeak 对象和一个 WhatCanMotion 对象。3. 向上类型转换是安全的、隐式的。当把 Human 对象的地址 &human 传递给这两个函数时, 编译器会自动将其向上转换为基类指针, 从而能够被函数接受。

技巧

多重继承最安全、最经典的使用场景就是**继承多个纯抽象接口类**。这样能清晰地展示一个类所具备的所有能力, 是实现“组合大于继承”设计思想的有力工具。

四、多态

4.1 多态的概念与条件 **必背**1. **定义**

- **多态**: 相同的代码逻辑, 在运行时能依据对象的实际类型产生不同行为的能力。
- **多态的必要条件**: 继承 + 虚函数 + (基类指针或基类引用)。这三个条件缺一不可。

2. 原理与推导

- **核心机制**: 通过基类指针/引用调用虚函数。编译期看到的是对基类虚函数的调用, 但运行时利用虚函数表机制 (晚绑定/动态绑定), 找到并执行实际指向的派生类中重写后的函数版本。
- **非多态情况**:
 - 通过**对象直接调用**: 无论函数是否为虚函数, 均在编译期确定调用版本 (早绑定), 不会发生多态。
 - 通过指针/引用调用**非虚函数**: 也在编译期绑定。

3. **解题步骤**

- **判断是否有多态行为**: 观察代码, 同时满足以下三点即为多态。
 - (a) class A 和 class B : public A 存在继承关系。

(b) A 中声明了 virtual 函数，且 B 中 override 了该函数。

(c) 调用形式是 `A* ptr = &bObj; ptr->virtualFunc();` 或 `A& ref = bObj; ref.virtualFunc();`;

4.2 动态多态与静态多态对比 高频

1. 对比与定义

特性	动态多态	静态多态
实现机制	继承 + 虚函数 (Virtual Functions)	模板 (Templates) + 函数重载 (Overloads)
绑定时期	运行时 (晚绑定)	编译时 (早绑定)
核心思想	运行时改变行为	编译时生成代码
性能	有虚函数调用的间接开销，稍慢	无运行时开销，高效
灵活性	灵活，可在运行时动态替换对象	编译后代码固定，不灵活
代码量	不增加二进制代码大小	为每种使用的类型生成一份代码，可能使二进制文件变大
侵入性	侵入式设计，必须继承自基类	非侵入式，只要类型支持模板内使用的操作即可

1. 易错点

- 混淆 C++ 中“多态”一词的狭义与广义。通常面试/考试中问“多态”，默认指动态多态。
- 误以为函数重载是多态。函数重载是编译时多态（静态多态）的一种。

2. 例题

例题

题目

分析程序运行结果，说明多态产生的效果。（课件 P29-P30 改编）

```
#include <iostream>

class Animal {
public:
    void action() { // 算法骨架, Template Method
        speak();
        motion();
    }
    virtual void speak() { std::cout << "Animal speak" << std::endl; }
    virtual void motion() { std::cout << "Animal motion" << std::endl; }
};

class Bird : public Animal {
```

```
public:
    void speak() override { std::cout << "Bird singing" << std::endl; }
    void motion() override { std::cout << "Bird flying" << std::endl; }
};

int main() {
    Bird bird;
    Animal* pBase = &bird;

    pBase->speak();          // 调用 (1)
    pBase->action();         // 调用 (2)

    Animal animal;
    animal.action();        // 调用 (3)
    return 0;
}
```

例题

思路

1. 调用 (1) 是典型的动态多态形式。2. 调用 (2) 的主体是 `action()` 函数，我们需要分析 `action` 内部调用的 `speak()` 和 `motion()` 的 `this` 指针是什么。3. 调用 (3) 是普通对象调用，`this` 指针指向基类对象自身。

解答

- 调用 (1): `pBase` 是基类指针，指向的是 `Bird` 对象，`speak` 是虚函数。满足多态条件，将调用 `Bird::speak`，输出 `Bird singing`。- 调用 (2): `pBase->action()` 内部，`this` 指针就是 `pBase`。在 `action` 中调用 `speak()` 和 `motion()`，等价于 `this->speak()` 和 `this->motion()`。由于 `this` 是基类指针，且指向 `Bird` 对象，调用的又是虚函数，所以仍会发生多态。因此输出 `Bird singing` 和 `Bird flying`。- 调用 (3): `animal` 是 `Animal` 对象。虽然 `action` 内部通过 `this` 调用虚函数，但 `this` 指向的是基类对象本身，不发生多态。输出 `Animal speak` 和 `Animal motion`。

技巧

在一个非虚成员函数内部调用虚函数，多态仍然会发生。关键在于 `this` 指针实际指向的是否是派生类对象。这正是模板方法模式的原理所在。

五、函数模板与类模板

5.1 函数模板与类型推导 必背

1. 定义

- 函数模板：一种用于创建与类型无关的函数的蓝图。定义时以 `template <typename T>`

开头，将函数中的类型用 `T` 替换。

```
template <typename T> // 或 class T
ReturnType funcName(ParamList) { /* 函数体 */ }
```

- **实例化**：编译器根据调用时传入的实参类型，自动生成一个特定类型的函数版本的过程。

1. 原理与推导

- **编译期机制**：模板是纯粹编译期的工具。编译器遇到模板定义时，不生成任何代码。只有遇到一个模板函数的调用时，编译器才根据实参类型推导出 `T`，然后生成并编译那份特定类型的代码。
- **推导失败与强制指定**：

```
template<typename T> T sum(T a, T b) { return a + b; }
sum(9, 2.1); // 编译错误：int和double二义性，无法推导T。
sum<int>(9, 2.1); // 通过，手动指定T为int，2.1被转换为int。
```

- **类型要求**：模板函数内部对类型 `T` 的操作（如 `+`, `>`, `<<`）会对传递给 `T` 的实际类型提出要求。如果类型不支持这些操作，编译器会报错。

1. 易错点

- **声明与定义分离问题**：函数模板（和类模板的成员函数）的声明和定义通常不能分开放置在 `.h` 和 `.cpp` 文件中，而必须全部放在头文件（`.h` 或 `.hpp`）中。原因是编译器在实例化时需要“看到”模板的完整定义。

2. 例题

例题

题目

下面的代码可以正确编译运行吗？如果不能，说明为什么。如果可以，输出什么？（课件 P38-P41 分析）

```
#include <iostream>
template<typename T>
T myMin(T a, T b) { return (a < b) ? a : b; }

int main() {
    std::cout << myMin(5, 3.75) << std::endl; // 调用(1)
    return 0;
}
```

例题

思路

1. 判断 `myMin(5, 3.75)` 能不能触发实例化。2. 若能，推导出的 `T` 是否唯一。

解答

1. 此代码无法通过编译。2. 原因：在调用 `myMin(5, 3.75)` 时，第一个参数 `5` 是 `int` 类型，第二个参数 `3.75` 是 `double` 类型。编译器试图推导模板参数 `T` 的类型，但发现两个实参类型不一致，产生二义性，无法推导，因此报错。3. 若要使其通过，可以手动指定模板参数类型，例如 `myMin<double>(5, 3.75);`。

技巧

函数模板要求用于推导同一模板参数 `T` 的所有实参类型必须严格匹配，否则编译器无法完成自动推导。遇到此类编译错误，优先检查参数类型是否一致，或显式指定模板参数类型。

5.2 类模板与非类型参数 **重点**

1. 定义

- **类模板**：与函数模板类似，用于创建与类型无关的类蓝图。所有成员函数在使用时都需要带上 `A<T>::` 的修饰。

```
template <typename T> class A {
    T data;
public:
    A(T d) : data(d) {}
    void print(); // 可在类外定义
};
// 类外定义成员函数
template<typename T>
void A<T>::print() { std::cout << data << std::endl; }
```

- **非类型参数**：模板参数也可以是具体的值，而不仅仅是类型。值参数的类型通常是整数、枚举、指针或引用，且在编译期必须是常量。

```
template<typename T, unsigned size> // size 是非类型参数
class MyArray {
    T data[size]; // 使用非类型参数指定数组大小
};
```

1. 原理与推导

- **实例化**：类模板的实例化必须在定义对象时显式指定模板参数。

```
MyArray<int, 10> arr1; // 创建了一个存储10个int的MyArray对象
```

- **编译期确定**：所有的模板参数，无论是类型参数还是非类型参数，都必须在**编译期**完全确定。这意味着非类型参数 `size` 必须是一个**常量表达式**（如字面值 10，或 `const int m = 5;`），而不能是运行时才能得到值的变量。

1. 易错点

对比项	类型参数	非类型参数
声明	typename T 或 class T	类型名 + 参数名，如 int N
实参	编译时可确定的数据类型，如 int, MyClass	编译时可确定的常量值，如 10, 'c'
错误示例	-	int n = 5; MyArray<char, n> a; (变量不行)
正确示例	MyArray<int, 5> a1;	const int sz = 5; MyArray<char, sz> a2; (常量可以)

1. 例题

例题

题目

指出并修正下面代码中的错误。（课件 P48 分析）

```
template<typename T, unsigned size>
class FixedArray {
    T data[size];
public:
    void printSize() { std::cout << "Size: " << size << std::endl; }
};

int main() {
    unsigned n = 10;
    FixedArray<int, n> fArr; // 错误行
    fArr.printSize();
    return 0;
}
```

例题

思路

1. 分析模板参数要求的性质（编译期常量 vs. 运行时变量）。2. 判断传入的参数 `n` 是否满足要求。

解答

1. 错误: `FixedArray<int, n>` 这行编译失败。2. 原因: `n` 是一个普通的 `unsigned` 变量, 其值在运行时才存在。而类模板 `FixedArray` 的非类型参数 `size` 要求在编译期就能确定。将运行时变量传递给编译期参数是非法的。3. 修正方法: 将 `n` 声明为 `const unsigned n = 10;`, 或者直接使用字面值 `FixedArray<int, 10>`。

技巧

判断传递给非类型模板参数的实参是否合法的核心标准是: 这个值在编译时不运行程序, 编译器能否唯一确定它? 如果能, 则合法; 否则非法。

本章小结

本章系统讲解了 C++ 中实现多态与复用的两大利器: 面向对象继承体系与泛型模板。

知识关系: 纯虚函数与抽象类定义了“接口”, 向下类型转换让我们能安全地从接口访问到实现, 而多态的核心就是通过基类接口来操纵不同派生类实现, 是前两者的应用与升华。函数/类模板则从另一个维度, 通过参数化类型来实现算法和容器的复用, 与继承虚函数形成“静多态”与“动多态”的互补。两者并非对立, 模板方法模式就是动静结合的典范。

核心结论:

1. 定义接口用抽象类, 所有未覆盖纯虚函数的派生类也是抽象类。
2. 向下类型转换永远优先使用 `dynamic_cast` 并检查结果。
3. 产生运行时多态的三大条件是: 继承、虚函数、指针或引用调用。
4. 模板的代码生成发生在编译期, 所有模板参数 (包括非类型参数) 必须在编译期确定。

必背公式:

- 纯虚函数: `virtual ReturnType func() = 0;`
- `dynamic_cast` 指针: `Derived* p = dynamic_cast<Derived*>(base_ptr); if(p != nullptr)...`
- 函数模板: `template<typename T> T func(...)`
- 类模板: `template<typename T> class MyClass {...};`

记忆 “接口用纯虚, 转换用 `dyna`, 多态看基指, 模板定编译。”